Công nghệ mã nguồn mở

ThS. Nguyễn Thị Mai Trang

Nội dung

Chương 1 Tổng quan về mã nguồn mở

Chương 2 Hệ điều hành mã nguồn mở

Chương 3 Lập trình SHELL

Chương 4 Lập trình với Python

Lập trình Shell

Chương 3

Nội dung chương 2

- Giới thiệu
- Cú pháp ngôn ngữ Shell
- Làm việc với chuỗi và văn bản
- Mảng
- Hàm

3.1. Giới thiệu

- HĐH cung cấp khả năng giao tiếp với kernel thông qua trình thông dịch trung gian → shell
- shell là một Command Line Interface (CLI) cho phép tương tác trực tiếp với hệ điều hành
- Chức năng giống “command.com” (DOS)
- Các lệnh trong và lệnh ngoài
- Cho phép kết hợp nhiều tiến trình :

  $ls –al | more

Giới thiệu (tt)

- Các loại Shell khác trên Linux
  - Bourne shell: shell đầu tiên, còn gọi là sh, do Steve Bourne tạo ra
  - C shell: cú pháp giống C, còn gọi là csh, đi kèm với BSD UNIX
  - Korn shell: còn gọi là ksh, kết hợp sh và csh, do David Korn tạo ra
  - Bash shell (Bourne Again SHell): Phiên bản cải tiến của sh, phổ biến nhất hiện nay và là mặc định trong Linux
  - Người dùng có thể chuyển đổi giữa các shell
    Ví dụ: chuyển từ bash sang csh, gõ lệnh:

    $ csh

Giới thiệu (tt)

- Sử dụng Shell như ngôn ngữ lập trình: có hai cách để viết chương trình điều khiển shell
- Cách 1: Gõ chương trình trực tiếp ngay dòng lệnh

```
$for file in *
>do
>  if grep -l 'main()' $file
>  then
>    more $file
>  fi
>done
```

  – Có thể viết liên tục các lệnh phân cách bởi dấu “;”)

```
$for file in *;do;if grep -l 'main()' $file; then;
more $file;fi;done
```

Giới thiệu (tt)

- Cách 2: viết các câu lệnh vào một tập tin, cấp quyền thực thi

```
# script tìm trong thư mục hiện hành #
các chuỗi chứa chuỗi "main()"
for file in *
do
  if grep -l 'main()' $file
  then
    more $file
  fi
done
exit 0
```

Giới thiệu (tt)

- Tạo tập tin shell script bằng một trình soạn thảo văn bản (thông thường, tập tin shell nên có phần mở rộng là .sh)
- Cấp quyền thực thi cho tập tin bằng lệnh:

  chmod +x tentaptin
- Thực thi tập tin:

  ./tentaptin

Cấp quyền và thực thi chương trình hello :

Ví dụ

```
$cd  /home/hv/baitap
$chmod +x hello
$./ hello
```

2.2. Cú pháp ngôn ngữ Shell

- Biến
- Chuyển hướng vào/ra
- Lệnh kiểm tra
- Biểu thức tính toán expr
- Cấu trúc điều khiển
- Danh sách
- Lấy về kết quả một lệnh

Biến

- Biến không cần phải khai báo trước khi sử dụng
- Có phân biệt chữ hoa, chữ thường
- Biến trong shell luôn có dạng chuỗi
- Gán trị cho biến:
  - Tenbien=giatri (không có khoảng trắng hai bên dấu =)
    - name=“hung”
    - number=100

Biến

- Lấy giá trị của biến: sử dụng $ trước tên biến
  - Ví dụ:

    tp=‘HaNoi’

    echo $tp

    →HaNoi
- Hiển thị giá trị của biến ra màn hình: sử dụng lệnh echo
  - echo ‘noi dung’
  - echo “noi dung”

Biến

- Phân biệt giữa cặp nháy đơn và nháy đôi
  - Nội dung nằm giữa cặp nháy đơn được xem là hằng chuỗi
    - Ví dụ:

```
echo ‘ Gia tri cua bien la $bien ‘

Kết quả hiện thị : Gia tri cua bien la $bien
```

  - Nội dung nằm giữa cặp nháy đôi có thể là hằng chuỗi và cả giá trị biến, muốn hiển thị các ký tự đặc biệt, đặt sau ‘\’

```
echo    ten=Dung
echo    “Su dung dau nhay kep”
echo    “Gia tri bien la $ten ”
echo    “Ky hieu tien la \$”
```

Ví dụ

```
Su dung dau nhay kep
Gia tri bien la Dung
Ky hieu tien la $
```

Kết quả hiển thị

Biến

- Nhập giá trị cho biến từ bàn phím
  - Cú pháp: read tenbien

Ví dụ

```
echo “Nhap vao ten cua ban “
read  ten
echo “Ten vua nhap la $ten”
```

Biến

- Biến môi trường:
  - Được định nghĩa sẵn, tên biến môi trường thường là chữ hoa

HOME	Chứa thư mục home của người dùng
PATH	Danh sách các thư mục tìm kiếm khi thực hiện các lệnh
PS1	Dấu nhắc lệnh (root: #, người dùng bình thường: $)
PS2	Dấu nhắc thứ cấp, thường là >
IFS	Dấu phân cách các trường trong danh sách chuỗi, thường là dấu khoảng trắng, tab và xuống hàng
PPID	Số ID của tiến trình cha trong SHELL
RANDOM	Số ngẫu nhiên
SECONDS	Thời gian làm việc tính theo giây

Biến

- Biến tham số:
  - Dùng để tiếp nhận tham số trên dòng lệnh cho việc xử lý
  - Sau các lệnh được gọi thường có các tham số, đó là giá trị các biến tham số của chương trình

$1, $2, $3	Giá trị các biến tham số thứ nhất, thứ 2.. tương ứng với các tham số từ trái sang phải trong dòng tham số.
$0	Tên tập tin lệnh gọi
$*	Danh sách tham số đầy đủ
$#	Tổng số tham số.
$$	Số tiến trình mà chương trình đang hoạt động

Chuyển hướng vào ra

- Tiến trình linux luôn gắn liền với các đầu xử lý dòng (stream) dữ liệu như:
  - Đầu vào chuẩn: stdin (0)
  - Đầu ra chuẩn: stdout (1)
  - Cơ sở dữ liệu lỗi: stderr (2)

>	Đầu ra hướng tới …
>>	Nối vào nội dung của …
<	Lấy đầu vào từ < …
<< word	đầu vào là ở đây …
2>	đầu ra báo lỗi sẽ hướng vào …
2>>	đầu ra báo lỗi hướng và ghi thêm vào …

- Ví dụ:

  date > file1.txt: lấy kết quả lệnh date ghi vào file1.txt

  ls

  cat < file1 > file2: lệnh cat nhận nội dung file1 ghi vào file2

Lệnh kiểm tra

- Sử dụng lệnh test hoặc [ ], cho phép kiểm tra 3 kiểu sau:
  - Kiểm tra chuỗi:

Phép so sánh	Kết quả
Chuoi1 = chuoi2	Đúng (true) nếu 2 chuỗi bằng nhau
Chuoi1 != chuoi2	Đúng nếu 2 chuỗi khác nhau
-n chuoi	Đúng nếu chuỗi "chuoi" không rỗng
-z chuoi	Đúng nếu chuỗi "chuoi" rỗng

Lệnh kiểm tra

– So sánh toán học:

Phép so sánh	Kết quả
bieuthuc1 –eq biethuc2	Đúng nếu bieuthuc1 bằng bieuthuc2
bieuthuc1 –ne biethuc2	bieuthuc1 không bằng bieuthuc2
bieuthuc1 –gt biethuc2	bieuthuc1 lớn hơn bieuthuc2
bieuthuc1 –ge biethuc2	bieuthuc1 lớn hơn hoặc bằng bieuthuc2
bieuthuc1 –lt biethuc2	bieuthuc1 nhỏ hơn bieuthuc2
bieuthuc1 –le biethuc2	bieuthuc1 nhỏ hơn hoặc bằng bieuthuc2

Lệnh kiểm tra

– Kiểm tra tập tin:

Phép kiểm tra	Kết quả
-d file	Đúng nếu tập tin là thư mục
-e file	tồn tại trên đĩa
-f file	là tập tin thông thường
-g file	có xác lập set-group-id trên file
-s file	có kích thước >0
-u file	có xác lập set-user-id
-r file	cho phép đọc
-w file	có phép ghi
-x file	cho phép thực thi

Lệnh kiểm tra

– Ví dụ:

```
if test -f hello.c
then
        . . .
fi
```

```
if  [-f hello.c ]
then
        . . .
fi
```

Lấy kết quả của một lệnh

- Cú pháp $(command)
  - Ví dụ:

```
echo Current directory is $PWD
echo It contents $(ls -a) files
exit 0
```

- Lệnh let: sử dụng không cần tiền tố $:

```
no1=4
no2=5
let result=no1+no2
echo $result
let no1++
let no1--
let no1+=6 (tương đương với let no1=no1+6)
let no2-=6 (tương đương với let no2=no2-6)
```

Lấy trị biểu thức với [ ], (())

- Cú pháp $[bieuthuc], $((bieuthuc))
  - Ví dụ:
    - result=$[no1 + no2]
    - result=$((no1 + 50))
- Có thể sử dụng tiền tố $ bên trong toán tử [ ]:
    - result=$[$no1 + 5]

Biểu thức tính toán expr

- Dùng để tính trị các biểu thức số nguyên
- expr được đặt giữa hai dấu `
- Trong biểu thức, các toán tử và toán hạng phải cách nhau bằng khoảng trắng
- Cho phép sử dụng các toán tử sau:
  - +, -, *, /, %
  - |, &
  - >, >=. <, <=, !=
- Ví dụ:
  - result=`expr 3 + 4`
  - result=$(expr $no1 + 5)

Sử dụng bc

- Được dùng trong tính trị các biểu thức, kể cả số thực
- Hỗ trợ định dạng xuất
- Ví dụ:
  - echo "4 * 0.56" | bc → 2.24
  - no=54
    result=`echo "$no * 1.5" | bc`
    echo $result
    →81.0

Sử dụng bc (tt)

- Định dạng số thập phân với bc:
  - echo "scale=2;3/8" | bc → 0.37
- Chuyển đổi cơ số với bc
  - no=100
    echo "obase=2;$no" | bc
    →1100100
  - no=1100100
    echo "obase=10;ibase=2;$no" | bc
    →100

Kết nối lệnh, khối lệnh

- Kết hợp và (and)
  - Cú pháp: lenh1 && lenh2 && lenh3
  - Trả về true nếu cả ba lenh1,2,3 đều đúng
  - Trả về false nếu một trong các lệnh sai
- Kết hợp hoặc (or)
  - Cú pháp: lenh1 || lenh2 || lenh3
  - Trả về true nếu một trong các lệnh đúng
  - Trả về false nếu cả ba lenh1,2,3 đều sai
- Khối lệnh: các lệnh nằm giữa cặp { }

Cấu trúc điều khiển

- Cấu trúc lựa chọn
  - if
  - if..elif
  - case
- Cấu trúc lặp
  - for
  - while
  - until
- Lệnh break, continue, exit

if

- Cú pháp:

```
if condition
then
  statements
else
  statements
fi
```

Ví dụ

```
echo "Is it morning? Please answer yes or no"
read timeofday
if [ $timeofday = "yes" ]; then
  echo "Good morning"
else
  echo "Good afternoon"
fi
```

if..elif

- if..elif:

Ví dụ

```
echo "Is it morning? Please answer yes or no"
read timeofday
if [ $timeofday = "yes" ]
then
  echo "Good morning"
elif [ $timeofday = "no" ]; then
  echo "Good afternoon"
else
  echo "Sorry, $timeofday not recognized. Enter yes or no"
exit 1
fi
exit 0
```

case

- Cú pháp:

```
case variable in
pattern [ | pattern] ...) statements;;
pattern [ | pattern] ...) statements;;
...
esac
```

Ví dụ

```
echo "Is it morning? Please answer yes or no"
read timeofday
case "$timeofday" in
  yes) echo "Good Morning";;
  no ) echo "Good Afternoon";;
  y ) echo "Good Morning";;
  n ) echo "Good Afternoon";;
  * ) echo "Sorry, answer not recognized";;
esac
```

Lệnh case (tt)

- Có thể gộp các điều kiện:

```
echo "Is it morning? Please answer yes or no"
read timeofday
case "$timeofday" in
   yes | y | Yes | YES ) echo "Good Morning";;
   n* | N* ) echo "Good Afternoon";;
   * ) echo "Sorry, answer not recognized";;
esac
```

Vòng lặp for

- Cú pháp

```
for variable in values
do
statements
done
```

  – Ví dụ:

```
for foo in apple banana 43
do
   echo $foo
done
```

```
for file in $(ls f*.sh); do
   lpr $file
done
```

for (tt)

- Có thể sử dụng vòng lặp for như cú pháp C

```
for (( variable assignment ; condition ; iteration))
    do
        .....
        repeat all statements between do and
        done until condition is TRUE
    done
```

Ví dụ

```
for (( i=0; i<=5; i++ ))
do
        echo $i
done
```

Vòng lặp while

- Cú pháp:

```
while condition
do
   statements
done
```

- Ví dụ:

```
echo "Enter password"
read trythis
while [ "$trythis" != "secret" ]; do
  echo "Sorry, try again"
  read trythis
done
```

```
foo=1
while [ "$foo" –le 20 ]
do
  echo "Here we go again"
  foo=$(($foo+1))
done
```

Vòng lặp until

- Cú pháp:

```
until condition
do
   statements
done
```

condition = true→thoát vòng lặp

Ví dụ

```
echo 'Please enter a number'
read n
until [ $n –lt 10 ]
do
   echo $n
   n='expr $n – 1`
done
```

Lệnh break, continue, exit

- break: thoát khỏi vòng lặp
- exit: thoát khỏi chương trình, trở về dấu nhắc lệnh
- continue: không thực hiện các lệnh còn lại trong vòng lặp mà quay lên thực hiện vòng lặp kế tiếp

- Viết lệnh có tên là thongke cho phép người dùng nhập vào các số nguyên khác 0 (nhập 0 để dừng). Thống kê và hiển thị kết quả cho biết số lớn nhất, nhỏ nhất, trung bình cộng của các số vừa nhập

2.4. Xử lý chuỗi, văn bản

- Sử dụng Regular expressions
- Xử lý chuỗi

Regular expressions

- Biểu thức chính quy, được dùng trong tìm kiếm chuỗi

biểu thức	Ý nghĩa	Ví dụ
^	bắt đầu của dòng	^tux→ đầu dòng là chuỗi tux.
$	Kết thúc của dòng	tux$ →cuối dòng là chuỗi tux.
.	Đại diện cho chỉ 1 ký tự	Hack. → sau Hack là một ký tự bất kỳ (Hack1, Hacki)
[ ]	So khớp với các ký tự trong [ ]	coo[kl] → cook hoặc cool
[^]	Ngoại trừ các ký tự trong [ ]	9[^01] → 92, 93,… ngoại trừ 91 và 90

Regular expressions (tt)

biểu thức	Ý nghĩa	Ví dụ
[m-n]	So khớp với các ký tựtừ m đến n	[1-5] →các ký tự số từ 1 đến 5
?	Ký tự trước ? Chỉ xuất hiện 0-1 lần	colou?r → color hoặc colour, không thể là colouur
+	Ký tự trước + phải xuất hiện ít nhất một lần	Rollno-9+ →Rollno-99 hoặc Rollno-9,… nhưng không thể là Rollno-
*	Ký tự trước * có thể xuất hiện từ 0 đến nhiều lần	co*l →cl, col, cool,…
( )	Nhóm ký tự trong ( ) được xử lý như một thực thể	ma(tri)?x → max hoặc matrix

Regular expressions (tt)

biểu thức	Ý nghĩa	Ví dụ
{n}	Phần tử đứng trước phải xuất hiện n lần	[0-9]{3} → ba ký tự số từ 0-9 liên tục
{n,}	Phần tử đứng trước phải xuất hiện ít nhất n lần	[0-9]{2,} → tối thiểu là hai ký tự số từ 0-9
{n, m}	Phần tử đứng trước phải xuất hiện từ n đến m lần	[0-9]{2,5} → từ 2 đến 5 ký tự số liên tục
|	Hoặc	Oct (1st | 2nd) → Oct 1st hoặc Oct 2nd.
\	Chỉ định ký tự đặc biệt như $, ^, ., *, +	a\.b → a.b (bắt buộc có ký tự dấu chấm (.) giữa a và b)

Regular expressions (tt)

- Ví dụ:
  - (?[a-zA-Z]+ ?)
  - [0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}
  - Hoặc [[:digit:]]{1,3}\.[[:digit:]]{1,3}\.[[:digit:]]{1,3}\.[[:digit:]]{1,3}

Xử lý chuỗi

- **Trả về chuỗi con trong chuỗi**
  - Cú pháp:
    - **${string:position}**: trả về chuỗi con của chuỗi string bắt đầu ở vị trí position.
    - **${string:position:length}**: trả về length ký tự trong chuỗi string bắt đầu ở vị trí $position.
  - Ví dụ:
    - string=”this is a substring test”
      substr=${string:10:9}
      echo $substr →substring
    - var="Welcome to the geekstuff"
    - echo ${var:15} → geekstuff

Xử lý chuỗi (tt)

- **Tìm và xóa chuỗi từ đầu chuỗi**: toán tử # và ##
  - ${string#*substring}: trả về chuỗi đã loại bỏ chuỗi ngắn nhất khớp với $substring từ đầu chuỗi $string
  - ${string##*substring}: trả về chuỗi đã loại bỏ chuỗi dài nhất khớp với $substring từ đầu chuỗi $string
    - Ví dụ:
      - s1="Hi Mary. How are you."
      - s2=${s1#* } →Mary. How are you
      - s3=${s1##* } →you

remove_first.sh

```
s='abc def fgef def 123'
s1='def'
s2=${s#*$s1}
s3=${s##*$s1}
echo "s2=$s2"
echo "s3=$s3"
```

student@ubuntu: ~

```
student@ubuntu:~$ ./code/remove_first.sh
s2= fgef def 123
s3= 123
```

Xử lý chuỗi (tt)

- Tìm và xóa chuỗi từ cuối chuỗi: toán tử % và %%
  - ${string%substring*}: trả về chuỗi đã loại bỏ chuỗi ngắn nhất khớp với substring từ sau chuỗi string.
  - ${string%%substring*}: trả về chuỗi đã loại bỏ chuỗi dài nhất khớp với substring từ sau chuỗi string
    - Ví dụ:
      - s1="Hi Mary. How are you."
      - s4=${s1% *} → Hi Mary. How are
      - s3=${s1%% *} →Hi

remove_last.sh

```
s='abc def fgef def 123'
s1='def'
s2=${s%$s1*}
s3=${s%%$s1*}
echo "s2=$s2"
echo "s3=$s3"
```

student@ubuntu: ~

```
student@ubuntu:~$ ./code/remove_last.sh
s2=abc def fgef
s3=abc
```

Xử lý chuỗi (tt)

- Lấy độ dài chuỗi: **${#string}**: trả về số ký tự của chuỗi $string
- Tìm kiếm và thay thế:
  - **${string/s1/s2}**
    - Trả về chuỗi đã được thay thế s1 bằng s2 tại vị trí tìm thấy đầu tiên.

```
s1="Hi Mary. How are you"
s2="Mary"
s3="Don"
s4=${s1/$s2/$s3}
echo "s4=$s4"
```

  - **${string//s1/s2}**
    - Trả về chuỗi đã được thay thế tất cả các s1 bằng s2

```
s="red - green - blue"
s1=${s//-/;}
echo "s1=$s1"
```

Xử lý chuỗi (tt)

- Tìm kiếm và thay thế (tt):
  - ${string/#s1/s2}
    - Trả về chuỗi đã được thay thế s1 bằng s2 nếu s1 có ở đầu chuỗi
  - ${string/%s1/s2}
    - Trả về chuỗi đã được thay thế s1 bằng s2 nếu s1 có ở cuối chuỗi

replace_firstlast.sh

```
s='def abc def fgef def 123 def'
s1='def'
s2='xyz'
s3=${s/#$s1/$s2}
s4=${s/%$s1/$s2}
echo "s3=$s3"
echo "s4=$s4"
```

student@ubuntu: ~

```
student@ubuntu:~$ ./code/replace_firstlast.sh
s3=xyz abc def fgef def 123 def
s4=def abc def fgef def 123 xyz
```

2.5. Mảng

- Mảng thông thường
- Mảng kết hợp

Mảng thông thường

- **Khai báo và khởi tạo**:
  - array_var=(1 2 3 4 5 6)
  - array_var[0]="test1“
    array_var[1]="test2“
    array_var[2]="test3“
    array_var[3]="test4“
    array_var[4]="test5“
    array_var[5]="test6“

arraydemo.sh

```
arr=(item0 item1 item2 item3 item4 item 5)
echo "Cac phan tu mang: ${arr[*]}"
echo "Cac phan tu mang: ${#arr[*]}"
echo "Phần tử có chỉ số 2: ${arr[2]}"
```

student@ubuntu: ~

```
student@ubuntu:~$ ./code/arraydemo.sh
Cac phan tu mang: item0 item1 item2 item3 item4 item 5
Cac phan tu mang: 7
Phần tử có chỉ số 2: item2
```

- **Truy xuất phần tử mảng**:
  - In một phần tử: echo ${array_var[2]} → 3
  - In tất cả các phần tử: $ echo ${array_var[*]}
    → test1 test2 test3 test4 test5 test6
- **Truy xuất số phần tử mảng**: $ echo ${#array_var[*]} → 6

Mảng kết hợp

- Là loại mảng mà trường index có thể là một chuỗi
- Khai báo:
  - $ declare -A ass_array
- Thêm phần tử vào mảng:
  - $ ass_array=([index1]=val1 [index2]=val2)
  - Hoặc: $ ass_array[index1]=val1
  - Ví dụ:
    - $ declare -A fruits_value
    - $ fruits_value=([apple]='100dollars' [orange]='150 dollars')

Mảng kết hợp

- Truy xuất nội dung phần tử mảng:
  - $ echo "Apple costs ${fruits_value[apple]}“
    → Apple costs 100 dollars
- Liệt kê các trường index trong mảng:
  - $ echo ${!array_var[*]}
  - Hoặc: $ echo ${!array_var[@]}
  - Ví dụ:
    - $ echo ${!fruits_value[*]} → orange apple

2.6. Hàm

- Cấu trúc hàm:

```
function_name ()
{
   statements
}
```

- Ví dụ:

```
func()
{
  echo "Function foo is executing"
}
echo "script starting"
func
echo "script ending"
```

Kết quả

```
script starting
Function foo is executing
script ending
```

Hàm (tt)

- Biến cục bộ (chỉ có hiệu lực bên trong hàm), khai báo dùng từ khoá local
- Biến toàn cục: khai báo không dùng từ khóa local (global)
- Phạm vi lưu trữ của biến cục bộ không còn hiệu lực khi hàm kết thúc

Hàm (tt)

- Ví dụ sử dụng biến cục bộ và toàn cục:

```
sample text="global variable"
foo() {
    local sample_text="local variable"
    echo "Function foo is executing"
    echo $sample_text
}
echo "script starting"
echo $sample_text
foo
echo "script ended"
echo $sample_text

exit 0
```

Kết quả

```
script starting
global variable
Function foo is executing
local variable
script ended
global variable
```

Hàm (tt)

- Hàm có thể trả về một giá trị
  - Trả về giá trị số: dùng lệnh return
  - Trả về chuỗi: dùng lệnh echo
  - Ví dụ:

```
foo( ) {
        ...
        return 0
        }
```

Hàm trả về số

```
foo() {
        echo "string value"
        }
. . .
x= $ ( foo )
```

Hàm trả về chuỗi

Hàm (tt)

- Hàm và cách truyền tham số:
  - shell không có cách khai báo tham số cho hàm như c, c++,…
  - Việc truyền tham số cho hàm tương tự truyền tham số trên dòng lệnh, ví dụ, để truyền tham số cho hàm foo, ta gọi như sau:
    - foo “param1” “param2” “param3”, …
    - Để lấy nội dung đối số, bên trong hàm, ta sử dụng các biến môi trường $*, $1, $2,…

Hàm (tt)

- Ví dụ truyền tham số cho hàm:

```
yes_or_no() {
echo "Is your name $* ?"
while true
do
  echo –n "Enter yes or no: "
  read x
  case "$x" in
    y | yes ) return 0;;
    n | no ) return 1;;
    * ) echo "Answer yes or no"
  esac
done
}
```

```
echo "Original parameters are $*"
if yes_or_no "$1"
then
  echo "Hi $1, nice name“
else
echo "Never mind"
fi
```

```
$ ./my_name.sh Rick Neil
Original parameters are Rick Neil
Is your name Rick ?
Enter yes or no: yes
Hi Rick, nice name
```

Hàm (tt)

- Lấy giá trị trả về của hàm

sum

```
sumof()
{
    return $(($1 + $2))
}
sumof $*
res=$?
echo "$1 + $2 = $res"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
student@student-pc:~/codes$ ./sum 10 15
10 + 15 = 25
student@student-pc:~/codes$
```

2.7. Các lệnh nội tại của shell

- Lệnh : (lệnh rỗng) tương đương với true
  - Ví dụ: kiểm tra nếu file fred tồn tại thì không làm gì cả

```
rm -f fred
if [ -f fred ]; then
    :
else
  echo file fred does not exist
fi
exit 0
```

- Lệnh exec: dùng để gọi một lệnh khác và gọi exit khi kết thúc lệnh
  - Ví dụ:

```
echo "Try to execute mc program"
exec mc
echo "you can not see this message !"
```

Các lệnh nội tại của shell (tt)

- Lệnh exit n: thoát khỏi shell và trả về mã lỗi n (0: không có lỗi)
- print (tương tự printf trong c), nhưng không hỗ trợ dạng số có dấu chấm động
  - Ký tự "\":

Chuỗi thoát (escape sequence)	ý nghĩa
\\	Cho phép hiển thị ký tự \ trong chuỗi
\a	Phát liếng chuông (beep)
\b	Xóa backspace
\f	Đẩy dòng
\n	Sang dòng mới
\r	Về đầu dòng
\t	Căn tab ngang
\v	căn tab dọc
\ooo	Kí tự đơn với mã là ooo

Các lệnh nội tại của shell (tt)

- print (tt)
  - Sử dụng ký tự "%" để định dạng xuất, các tham số của lệnh print cách nhau một khoảng trắng

Kí tự định dạng	Ý nghĩa
d	Số nguyên
c	Ký tự
s	chuỗi
%	hiển thị ký hiệu %

- Ví dụ

```
$ printf  "Your name %s. It is nice to meet you \n" NV An
Your name NV An. It's nice to meet you .

$ printf "%s  %d\t  %s" "Hi There" "15" "people"
Hi There 15 people
```

Các lệnh nội tại của shell (tt)

- Lệnh set
  - Dùng để gán giá trị cho các biến môi trường $1, $2, $3,… bằng cách loại bỏ các khoảng trắng không cần thiết và đặt nội dung của chuỗi truyền cho nó vào các biến tham số

```
$ set This is parameter
$echo $ 1
This
$echo $3
parameter
```

Ví dụ

Các lệnh nội tại của shell (tt)

- Lệnh set (tt)

```
echo Current date is  $(date)
set $(date)
echo The month is $2
echo The year is $6
exit 0
```

Kết quả kết xuất

```
$./set_use.sh
Current date Fri March 13      16:06:16 EST 2001
The month is March
The year is 2001
```

Các lệnh nội tại của shell (tt)

- Lệnh unset: loại bỏ biến khỏi shell

```
foo="Hello World"
echo $foo
unset foo
echo $foo
```

Q/A?